



The Native Object Store and Its POSIX Projection: Measured Throughput, Latency, and the Limits of Filesystem Emulation

Zach Kelling

Hanzo AI

Revision 1

The Native Object Store and Its POSIX Projection: Measured Throughput, Latency, and the Limits of Filesystem Emulation

Zach Kelling*

Hanzo Industries Inc (Techstars '17)

Los Angeles, California

<https://hanzo.ai>

Revision 1

Abstract

We report measurements of `hanzoai/s3`, the native Hanzo object store, and of the POSIX filesystem it projects through a Container Storage Interface (CSI) driver whose control path speaks the native ZAP transport rather than gRPC. All figures are taken from the production deployment, in-cluster, against authenticated requests.

The object store performs as an object store should: 265.9 MiB/s on 16 MiB reads, 12.81 ms median metadata lookups, and stable latency under mixed concurrent load. The filesystem projection sustains 287 MB/s sequential writes and 685 MB/s sequential reads.

Its metadata plane does not follow. The mount performs *27 file creations per second* and *201 stats per second*, three orders of magnitude below local block storage. We quantify the consequence on a real metadata-bound workload—a Git repository—and find slowdowns of $27\times$ to $278\times$. We therefore argue against the common instinct to relocate a Git forge onto an object-store-backed mount, and state the boundary we believe generalises: **project an object store into POSIX for bulk data, never for namespaces**.

We also document three defects found while bringing this path to production—two in connection-pooled RPC and one silently destroying data during metadata restore—because each was invisible to the tests that would ordinarily be trusted, and the reasons they were invisible are reusable.

1 System Under Test

`hanzoai/s3` is a fork of the SeaweedFS lineage that has replaced its filer RPC surface with ZAP, the Hanzo capability transport: 28 unary and 5 streaming methods answer over `transport.ListenStream` on the port historically named the “gRPC port”. No protobuf framing remains on that path. The measured build is `v1.0.4`.

The metadata store is `luxdb`, a wrapper over `github.com/luxfi/database` with a `zapdb` backend. This is the sole store compiled into the binary; the SeaweedFS `leveldb2` store was re-

moved. That removal is consequential and we return to it in Section 7.

The POSIX projection is `hanzoai/s3-csi`, a CSI driver forked so that its filer-facing half speaks ZAP. A volume is a directory beneath the filer root, so volumes are `ReadWriteMany` by construction. The kubelet-facing half remains CSI gRPC, which is the Kubernetes contract rather than a design choice.

1.1 Deployment Shape, Stated Honestly

It is worth being exact about durability, because the architecture is often described more ambitiously than the deployment warrants.

- Volume replication is configured "000": **one copy of every volume**. There is no redundancy at the store layer.
- The topology contains **one** volume server and one master member.
- The Deployment is **Recreate**, **replicas: 1**, bound to a single 700 GiB **ReadWriteOnce** block volume.
- **luxfi/consensus** is **not** a dependency of this service. No consensus protocol participates in its durability.

Durability therefore rests entirely on the underlying cloud block device and its snapshots. The store is not highly available: a roll is an outage, and a lost volume is a restore-from-backup event. We state this plainly because a paper that implied otherwise would be describing a system we do not run. Introducing replication, or placing the metadata plane under a consensus protocol, remains future work rather than shipped behaviour.

2 Method

Measurements were taken from pods inside the same cluster as the store, to exclude ingress and WAN effects. Object-store figures use **SigV4**-signed requests through the service DNS name; the driver and the store were the production instances serving live traffic. Each latency figure is the mean of ten repetitions; metadata figures are means over 20–500 operations. Filesystem figures use **dd** with **conv=fsync** for writes.

The Git comparison runs an identical script against the CSI mount and against the pod’s local disk, so the delta isolates the storage path.

Object size	PUT (ms)	GET (ms)	GET (MiB/s)
1 KiB	38.93	15.67	0.1
64 KiB	18.21	17.10	3.7
1 MiB	36.46	20.30	49.3
16 MiB	244.80	60.17	265.9

Table 1: Signed request latency, in-cluster, mean of 10.

3 Object Store Results

Metadata lookups (**HEAD** on an existing object) average 12.81 ms over 20 repetitions. Latency is dominated by a fixed per-request cost of roughly 15 ms until objects exceed 1 MiB, after which transfer dominates and throughput climbs to 265.9 MiB/s.

Under a sustained mixed load—eight rounds of four concurrent 1000-key **LIST** operations interleaved with **PUT/GET/DELETE**—the store completed every operation with no failures and unary latency flat between 7.5 and 21 ms. This load shape is precisely what exposed the transport defect of Section 6.3, and its stability is the evidence that the defect is resolved.

4 Filesystem Projection Results

Operation	Block size	Rate
Sequential write	1 MiB	188 MB/s
Sequential write	4 MiB	287 MB/s
Sequential read	1 MiB	651 MB/s
Sequential read	4 MiB	685 MB/s
File creation	—	27 files/s
stat	—	201 stats/s

Table 2: CSI mount, 20 GiB RWX volume.

The bulk-data path is strong and would satisfy most streaming workloads. The namespace path is not: creating a file costs approximately 37ms and **stat** approximately 5ms. Both are network round trips to the filer with no client-side namespace cache that survives them.

5 The Metadata Cost, Made Concrete

Aggregate rates understate how punishing this is, because real workloads issue metadata operations in dependent chains rather than in parallel. We therefore measured Git, which is metadata-bound almost by definition: loose objects, refs, and index updates are small files created and stat’ed in sequence.

Operation	Local	CSI mount	We regard this as the paper’s main result, and note it inverts the plan we began with. The prior reasoning—that the code was already correct and only the mount was missing—was true about the code and wrong about the workload.
init + 200 files + commit	1617 ms	43496 ms	27×
clone	83 ms	23044 ms	278×
gc -aggressive	162 ms	9699 ms	60×
log + status	12 ms	2857 ms	238×

Table 3: Identical Git workload, same pod, two storage paths.

A 278-fold penalty on `clone` is not a tuning problem. It is the arithmetic of thousands of dependent round trips at 5–37 ms apiece.

5.1 Consequence for the Git Forge

The motivating goal of this work was to relocate a 22 GiB Git forge (329 bare repositories) off a single `ReadWriteOnce` volume—the constraint pinning the control-plane binary to one node—onto shared storage. The forge’s storage layer is an `osfs` rooted at a data directory, so the change appeared to require no code at all: mount the directory elsewhere.

The measurements say do not. Git’s heavy paths (clone/fetch serve, push receive) shell out to the `git` CLI against an OS path and stream multi-gigabyte packfiles, which this mount serves well. But every other Git operation is metadata, and metadata is where the projection collapses.

The honest conclusions are:

1. **Do not place the live Git forge on this mount.** The 238× penalty on `log/status` alone would be user-visible on every request.
2. **The object store is the right home for packfiles**, which are large, sequential, and

immutable—exactly the shape it serves at 265.9 MiB/s.

3. **Decouple the two problems.** Single-node pinning is a *placement* problem; solving it with a filesystem projection imports a metadata cost the workload cannot absorb. A Git-native object backend, or replication at the block layer, addresses placement without paying it.

We regard this as the paper’s main result, and note it inverts the plan we began with. The prior reasoning—that the code was already correct and only the mount was missing—was true about the code and wrong about the workload.

6 Defects Found En Route

Three defects had to be fixed before any of the above could be measured. Each is reported because each defeated the check that should have caught it.

6.1 Promise identifiers scoped to the client, not the connection

Generated ZAP clients each carried a private `rpc.Session` numbering calls from 1, while the connection pool shares one connection per peer address and constructs a fresh client per call. Two concurrent calls therefore both claimed promise 1; the second registration evicted the first from the connection’s in-flight table, and the evicted response was discarded on arrival. The caller blocked for the lifetime of its context—at service start-up, `context.Background()`.

The invariant: **a promise identifier is the connection’s demultiplexing key, so the connection must allocate it.** Stream identifiers already did.

6.2 Metadata restore that dropped every file

The filer’s `meta.load` created directories inline and files on goroutines, and returned on end-of-input without waiting for them. The caller closes the connection on return, so in-flight writes died

against a torn-down connection—and the command reported success. A restored tree came back with its directory structure intact and *no files*.

This is the failure mode one discovers during a disaster recovery. It survived because the only signal was the absence of data nobody had yet tried to read.

6.3 Head-of-line blocking on pooled connections

Inbound stream frames were delivered from the connection’s read loop with a blocking send into a 16-slot buffer. A consumer slower than its producer filled that buffer, stalled the read loop, and with it every other stream and every unary reply on the connection. Pooling turned a throughput concern into an outage: one large LIST froze every request in the process.

It presented as an authentication fault, because unauthenticated requests are rejected before reaching the filer and continued answering in 5 ms. Diagnosis required a goroutine dump from the stalled process; no log line existed, because the code was not failing, only waiting.

The fix queues per stream and wakes the reader, so delivery never blocks the read loop; a bound fails one stream rather than the connection everyone shares.

6.4 What the three have in common

Two of the three were latent properties of *connection pooling*—a sharing optimisation that converts an isolated stall into a global one—and all three were invisible to unauthenticated smoke tests, to fresh-dataset tests, and to green unit suites. The tests that found them were: an adversarial reproduction with a deliberately slow consumer; a restore exercised against data written by the *previous* release; and a goroutine dump taken while production was hung.

7 Store Migration

Because luxdb is the only store compiled in, a version upgrade against an existing leveldb2 data

directory starts with an **empty namespace**—no error, no crash, every bucket invisible. The two layouts coexist in one directory (leveldb2 in numbered sub-directories, zapdb in top-level files), which is what makes the migration reversible.

The production migration moved 50,006 directories and 68,646 files, and all 45 buckets returned. Reversal is an image change; only writes made in the new store are lost. The upgrade path is therefore: export, roll, import, verify with a *signed* operation against *pre-existing* data—the last qualifier being the one that matters, since unauthenticated and freshly-created-object checks pass while the store is broken.

8 Conclusion

The native object store is fast, and its POSIX projection is fast for bulk data and slow for namespaces by three orders of magnitude. That asymmetry is not an implementation defect to be optimised away; it is the cost of emulating a filesystem over a network metadata service, and it decides which workloads may cross the boundary.

Bulk, sequential, immutable data belongs in the object store. Namespaces with dependent metadata chains—Git foremost—do not, and should not be moved there merely because their storage layer makes it syntactically easy.